

AUTOMATION THINKING

E-Commerce Automation Testing Project using Python & Selenium

Project Index / Structure

1. Introduction
2. Objective of Automation Testing
3. Application Update Scenarios
4. Automation Testing Requirements
5. Tools & Technologies Used
6. Selenium Automation Practice Platform
7. Automation Test Scenarios
8. Python Automation Scripts
9. Test Execution Process
10. Screenshots of Execution Results
11. Validation & Expected Results
12. Challenges Faced
13. Conclusion

Tools & Technologies Used

Programming Language

- Python

Automation Tool

- Selenium WebDriver

IDE / Code Editor

- VS Code

Browser

- Google Chrome

Browser Driver

- ChromeDriver

Testing Platform

- [SauceDemo](#)

Additional Libraries

- webdriver-manager
- pytest (optional)

Short Project Introduction

Introduction

This project focuses on automating important testing workflows of a web application using Selenium WebDriver and Python. The purpose of automation testing is to ensure that every application update is verified automatically and reduces repetitive manual testing efforts.

The automation scripts validate important functionalities such as:

- Login functionality
- Invalid login validation
- Product/order creation
- Checkout workflow
- Data validation

The practice automation environment used for this project is [SauceDemo](#), which simulates real-world e-commerce testing scenarios.

Automation Scenarios Included

Automated Test Cases

- Valid Login Automation
- Invalid Login Validation
- Add Item to Cart
- Remove Item from Cart
- Checkout Process Validation
- Order Completion Validation

Output Deliverables

Project Deliverables

- Python Selenium Scripts
- Execution Screenshots
- Test Result Validation
- Bug Observation Notes
- Automation Flow Explanation

Introduction

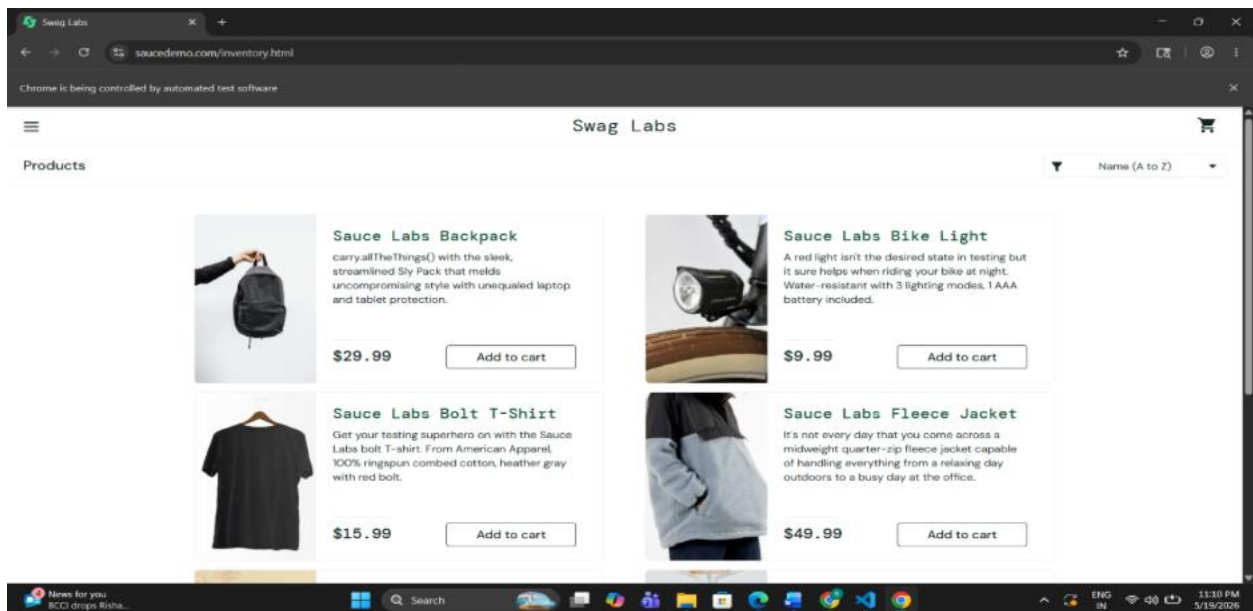
Automation testing is used to automatically verify important application functionality after every system update. In Medirack-like healthcare systems, automation helps ensure that critical workflows continue working correctly after UI or backend changes.

Example updates where automation is useful:

- OCR screen redesign
- Bill UI modification
- Payment upload changes
- Notification logic updates

Automation testing verifies:

- Login functionality
- Order creation process
- Data saving functionality
- Admin visibility of updated records



Selenium Automation Practice

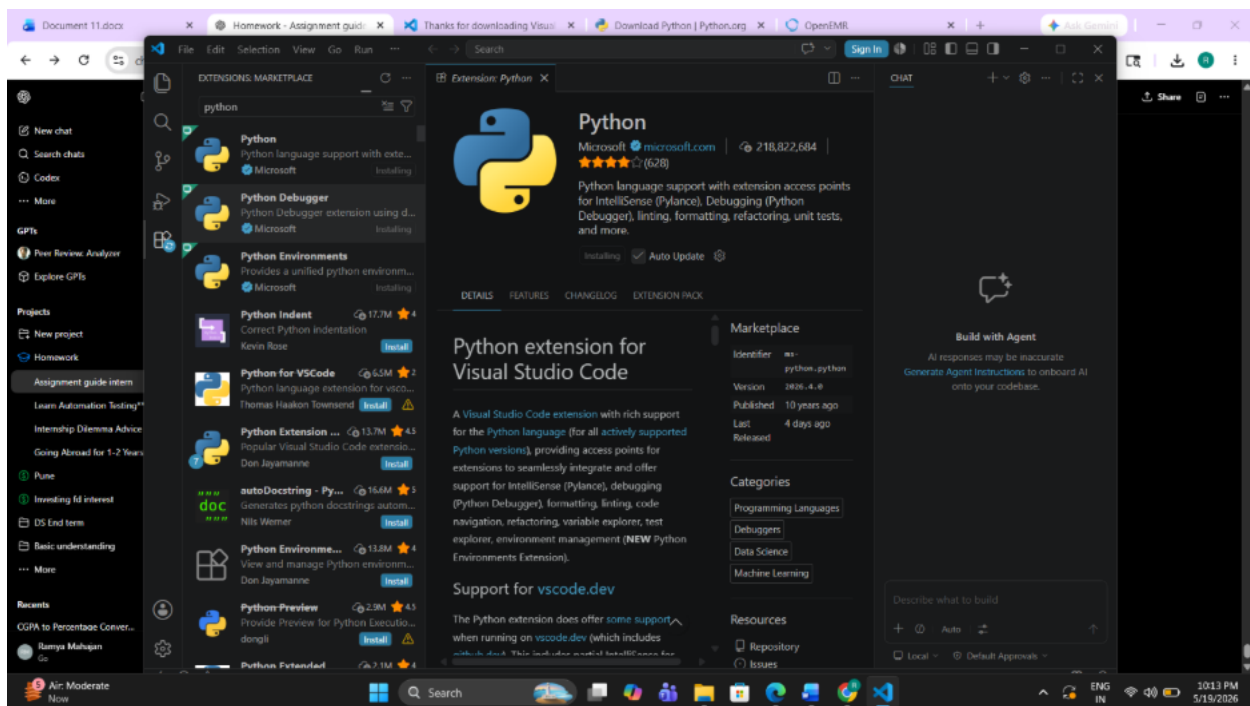
Practice Platform

[SauceDemo Official Website](#)

Tool Stack Used

- Python
- Selenium WebDriver
- Visual Studio Code (VS Code)
- Google Chrome Browser

(VS Code installed with Python extension)



1. Login Automation Script

Purpose

To verify successful login functionality using valid credentials.

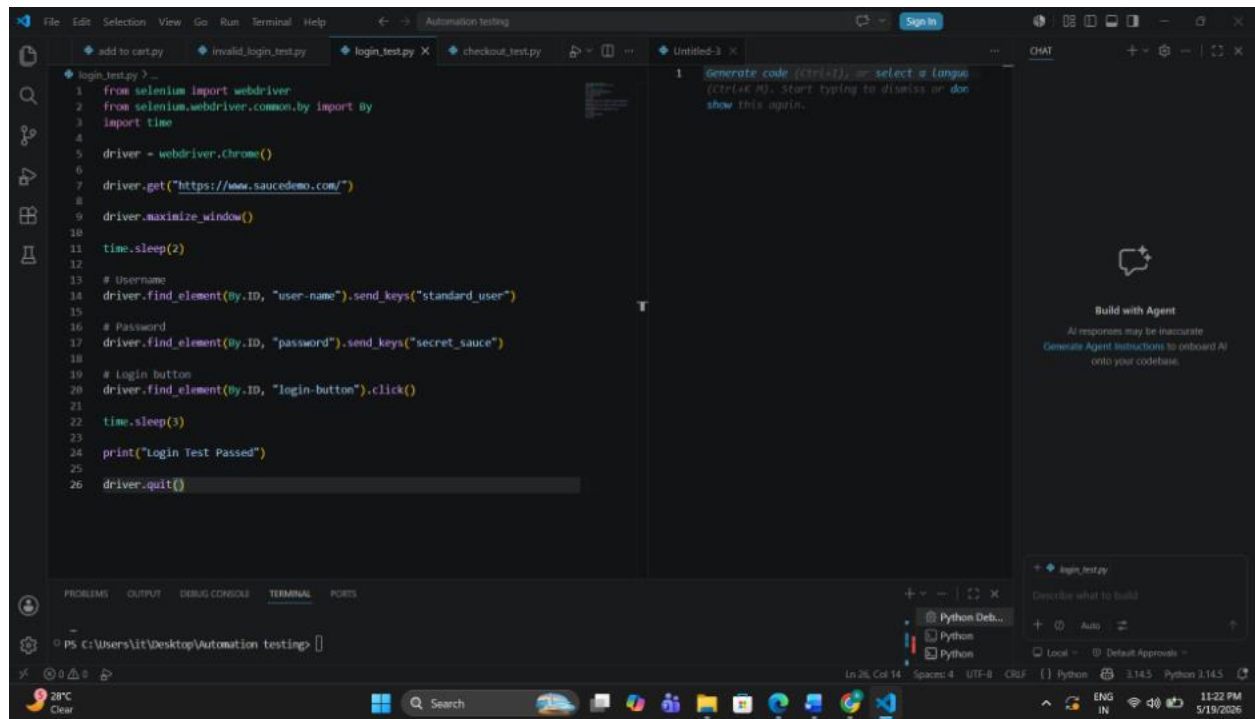
Script Validation

- Login page loads correctly
- Username field accepts input
- Password field accepts input
- Login button functions correctly
- User successfully enters application

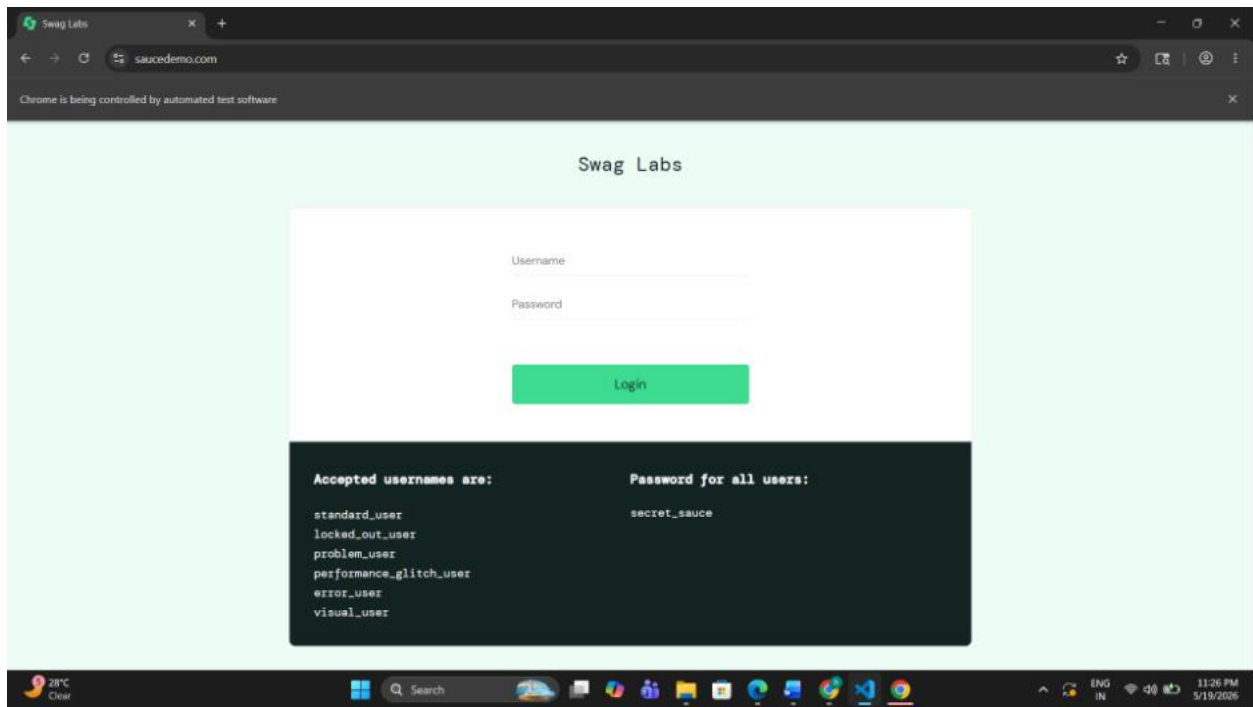
Result

Login executed successfully and inventory page opened.

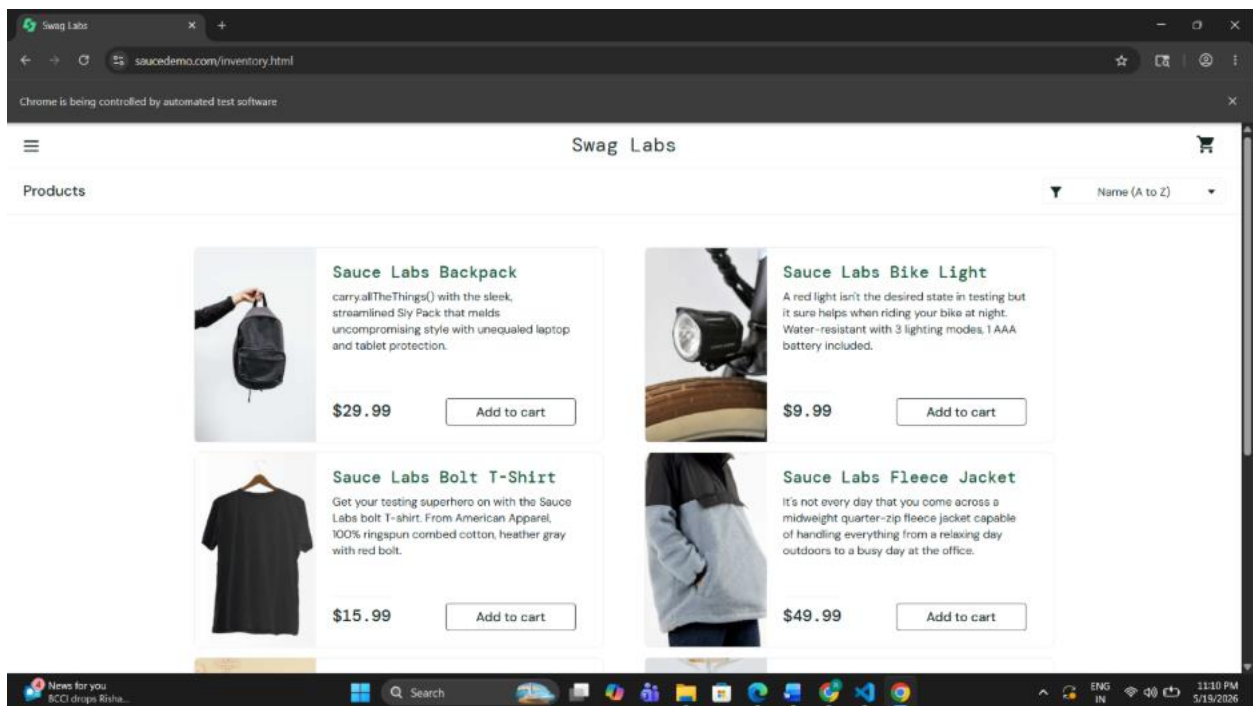
(VS Code showing `login_test.py` code)



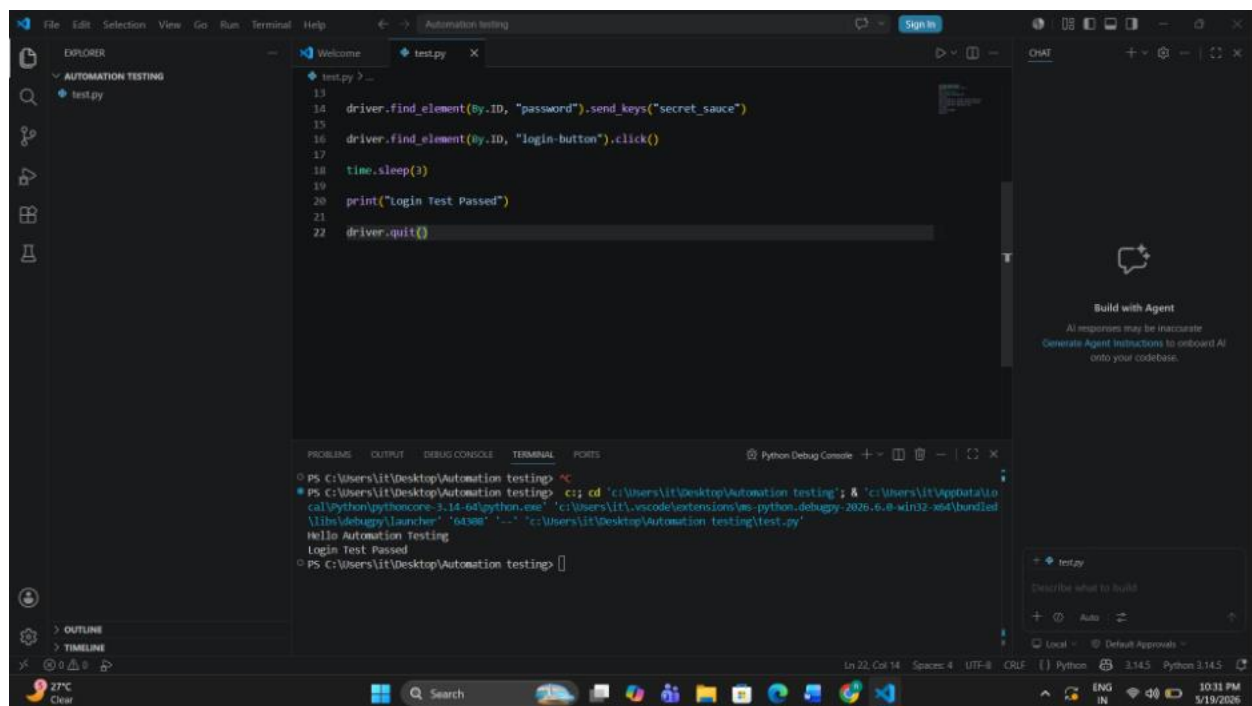
(Chrome browser automatically opening SauceDemo)



(Successful login inventory page)



(Terminal showing Login Test Passed)



The screenshot displays the Visual Studio Code interface with a Python script named `test.py` in the editor. The script uses Selenium to automate a login process. The terminal at the bottom shows the command to run the script, which executes successfully, printing "Hello Automation Testing" and "Login Test Passed".

```
13
14 driver.find_element(By.ID, "password").send_keys("secret_sauce")
15
16 driver.find_element(By.ID, "login-button").click()
17
18 time.sleep(3)
19
20 print("Login Test Passed")
21
22 driver.quit()
```

```
PS C:\Users\litt\Desktop\Automation testing>
PS C:\Users\litt\Desktop\Automation testing> cd 'c:\Users\litt\Desktop\Automation testing'; & 'c:\Users\litt\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'c:\Users\litt\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\lib\debugpy\launcher' '64888' '-' 'c:\Users\litt\Desktop\Automation testing\test.py'
Hello Automation Testing
Login Test Passed
PS C:\Users\litt\Desktop\Automation testing>
```

2. Invalid Login Automation Script

Purpose

To verify that invalid users are rejected by the system.

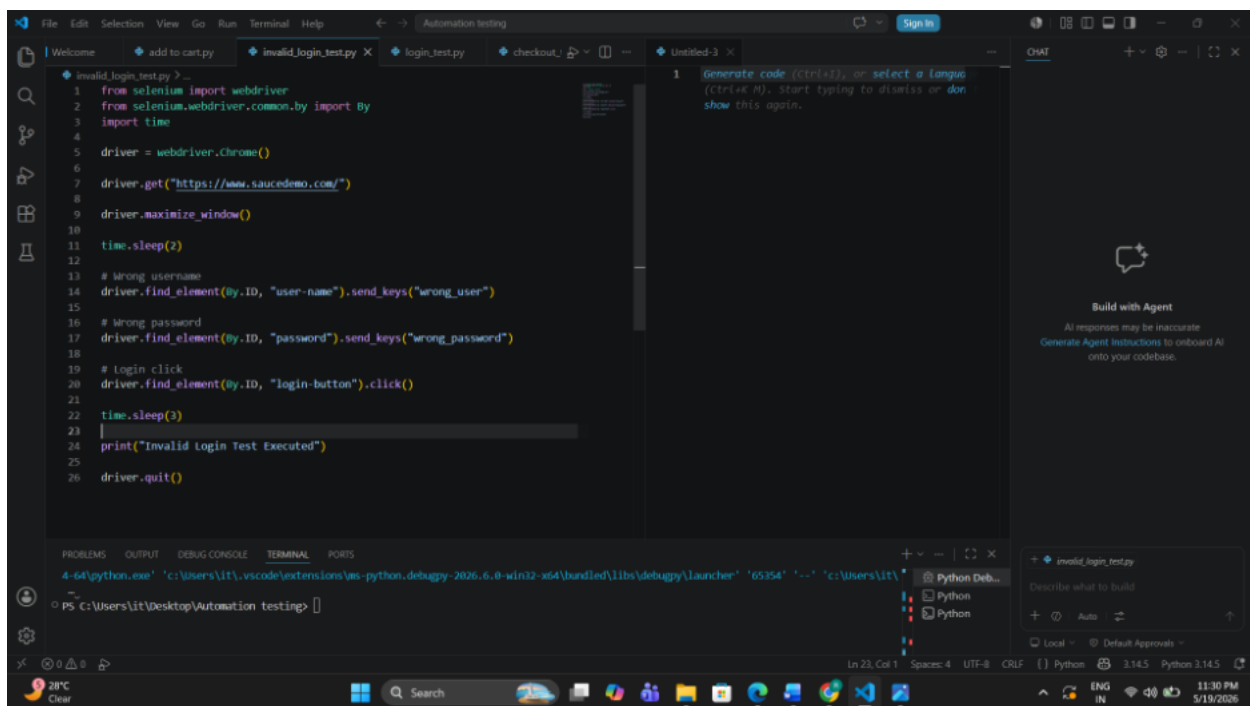
Script Validation

- Invalid credentials are blocked
- Authentication validation works correctly
- Unauthorized access is prevented
- Error handling functions properly

Result

System rejected invalid login credentials successfully.

(VS Code showing `invalid_login_test.py` code)

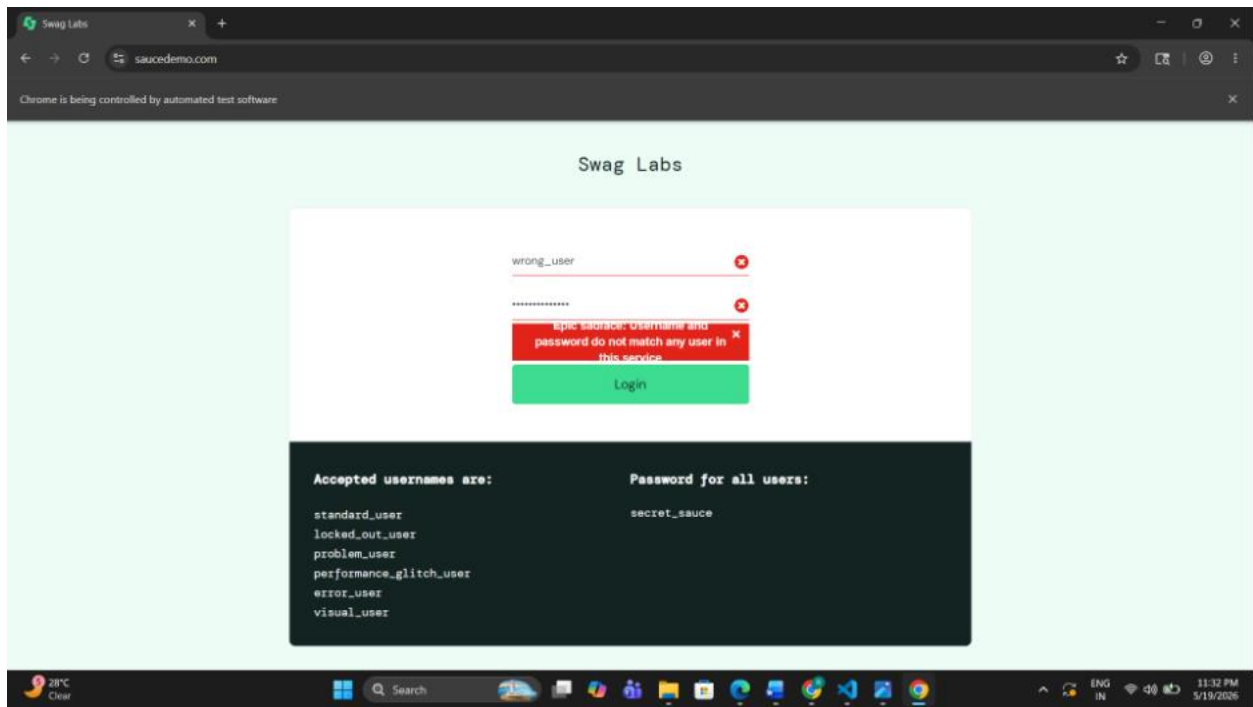


```
1 from selenium import webdriver
2 from selenium.webdriver.common.by import By
3 import time
4
5 driver = webdriver.Chrome()
6
7 driver.get("https://www.saucedemo.com/")
8
9 driver.maximize_window()
10
11 time.sleep(2)
12
13 # wrong username
14 driver.find_element(By.ID, "user-name").send_keys("wrong_user")
15
16 # wrong password
17 driver.find_element(By.ID, "password").send_keys("wrong_password")
18
19 # login click
20 driver.find_element(By.ID, "login-button").click()
21
22 time.sleep(3)
23
24 print("Invalid Login Test Executed")
25
26 driver.quit()
```

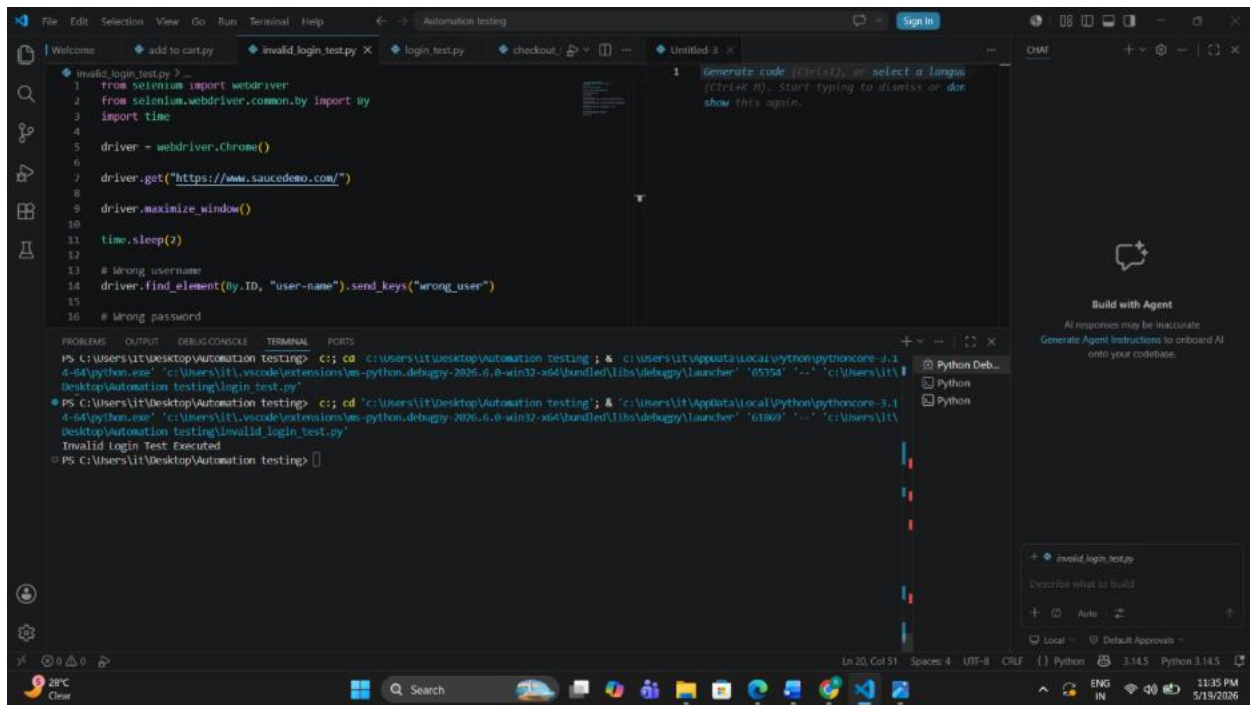
4-64\python.exe "c:\Users\lit\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher" '65354' '-' 'c:\Users\lit\...' Python Deb...

PS C:\Users\lit\Desktop\Automation testing>

(Login attempt with wrong credentials)



(Terminal showing Invalid Login Test Executed)



3. Add Item Flow Automation Script

Purpose

To verify whether products can be added into the shopping cart successfully.

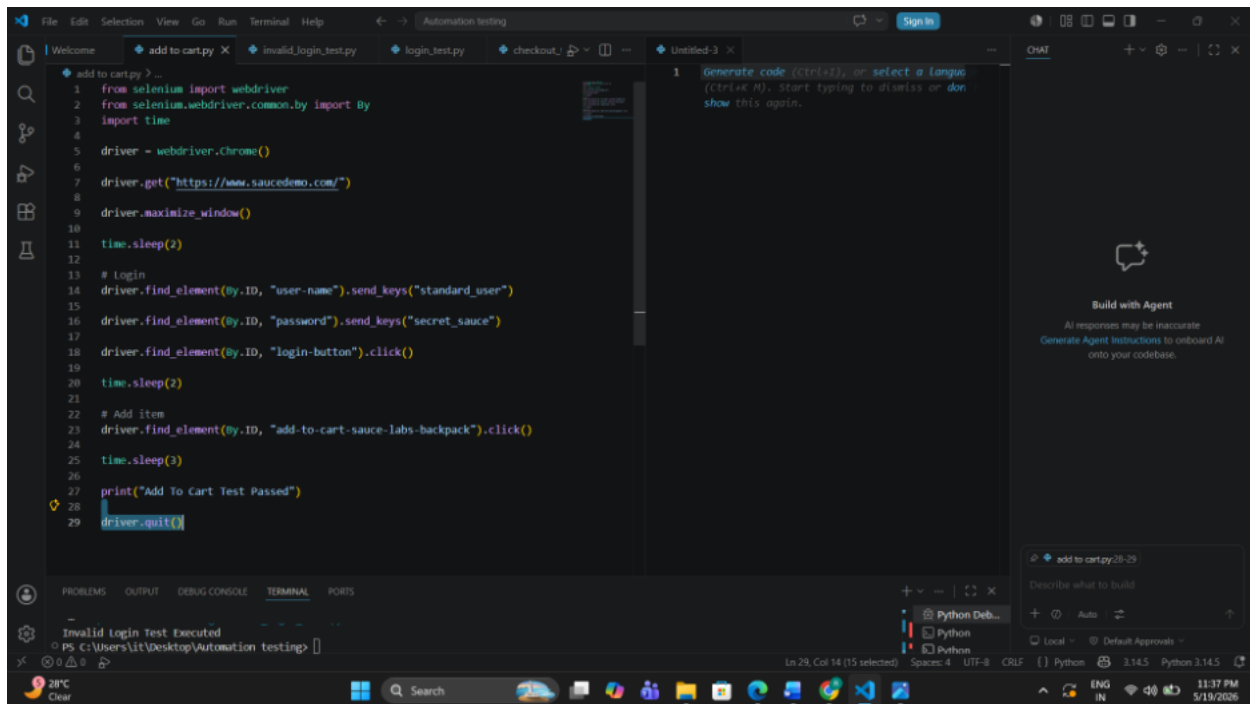
Script Validation

- Product inventory page loads properly
- Add to Cart button functions correctly
- Cart updates successfully
- Selected item is added properly

Result

Product was added into cart successfully.

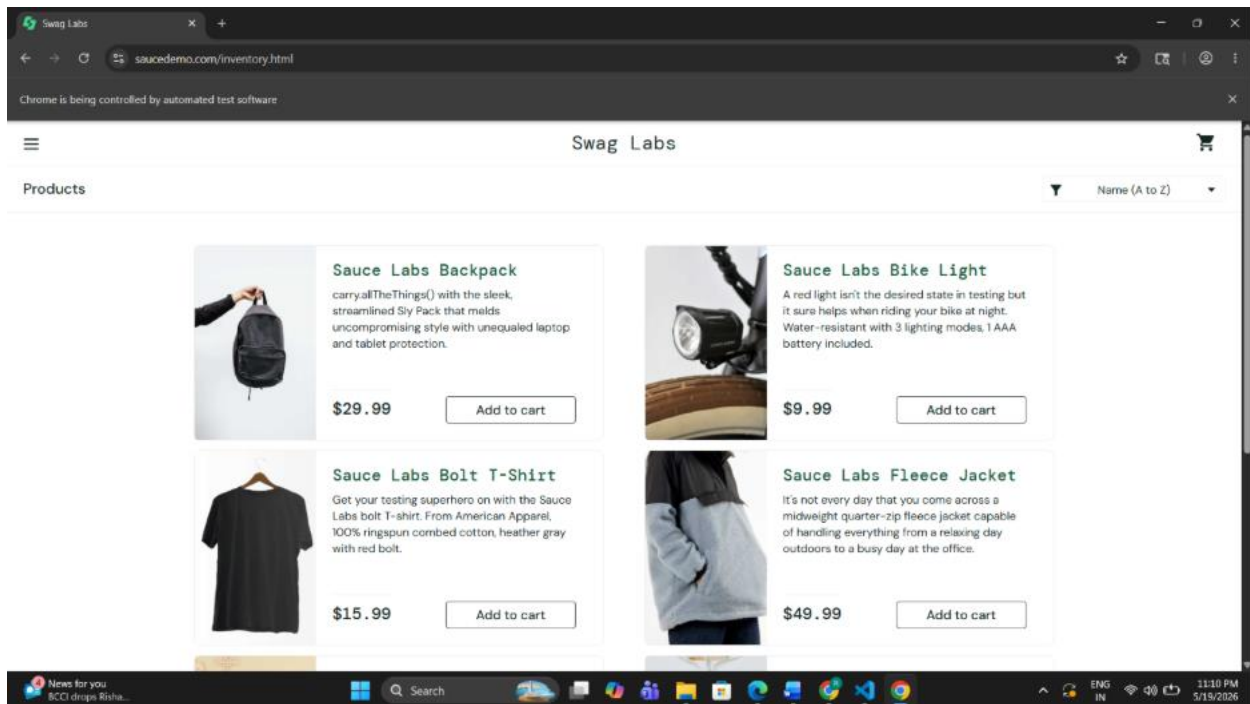
(VS Code showing add_to_cart.py code)



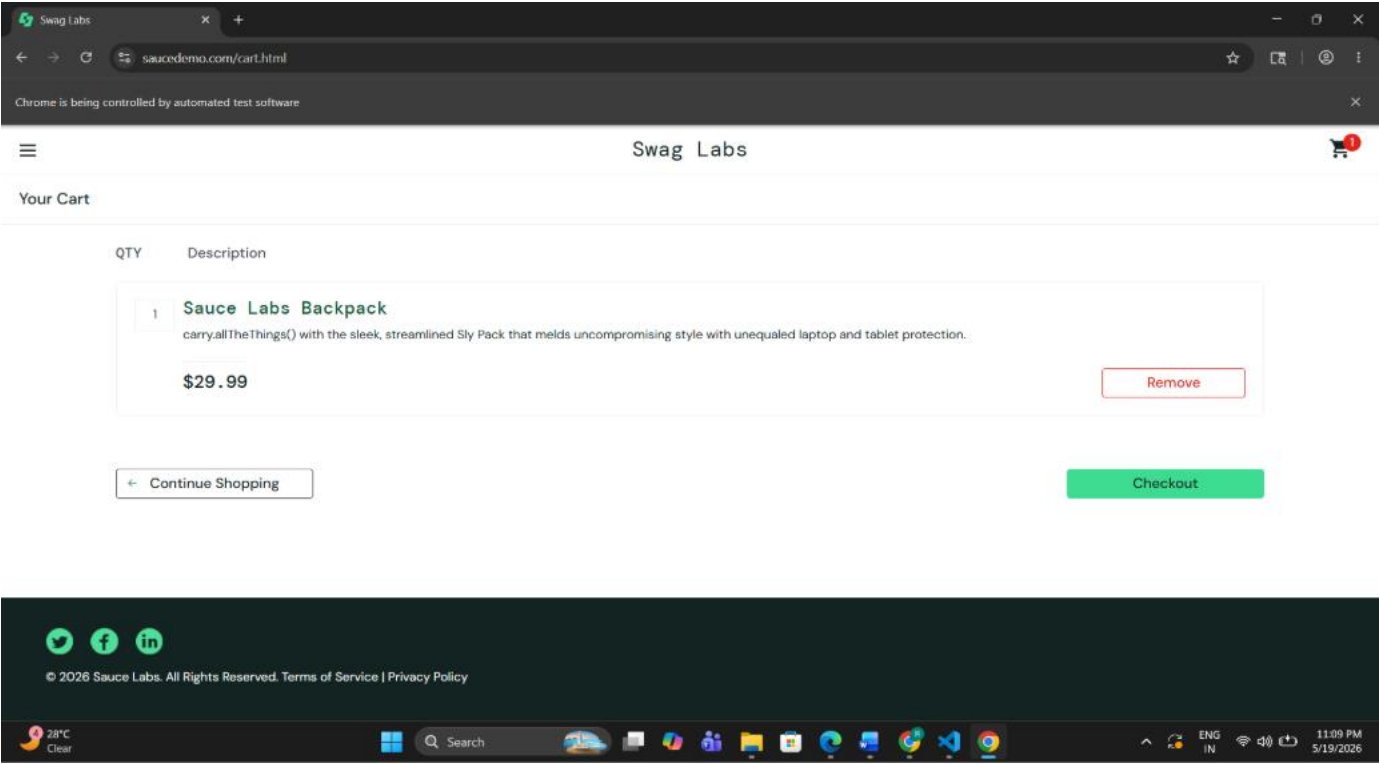
The screenshot shows the Visual Studio Code editor with a file named 'add_to_cart.py' open. The code is a Selenium script for automation testing. It imports 'webdriver' from 'selenium', sets up a Chrome driver, navigates to 'https://www.saucedemo.com/', maximizes the window, and performs login actions. After logging in, it clicks on the 'add-to-cart-sauce-labs-backpack' button, sleeps for 3 seconds, and prints 'Add To Cart Test Passed' before quitting the driver. The terminal at the bottom shows the command 'Invalid login Test Executed' and the file path 'C:\Users\lit\Desktop\Automation testing\'. The status bar indicates 'Ln 29, Col 14 (15 selected)' and 'UTF-8 CRLF'.

```
1 from selenium import webdriver
2 from selenium.webdriver.common.by import By
3 import time
4
5 driver = webdriver.Chrome()
6
7 driver.get("https://www.saucedemo.com/")
8
9 driver.maximize_window()
10
11 time.sleep(2)
12
13 # Login
14 driver.find_element(By.ID, "user-name").send_keys("standard_user")
15
16 driver.find_element(By.ID, "password").send_keys("secret_sauce")
17
18 driver.find_element(By.ID, "login-button").click()
19
20 time.sleep(2)
21
22 # Add item
23 driver.find_element(By.ID, "add-to-cart-sauce-labs-backpack").click()
24
25 time.sleep(3)
26
27 print("Add To Cart Test Passed")
28
29 driver.quit()
```

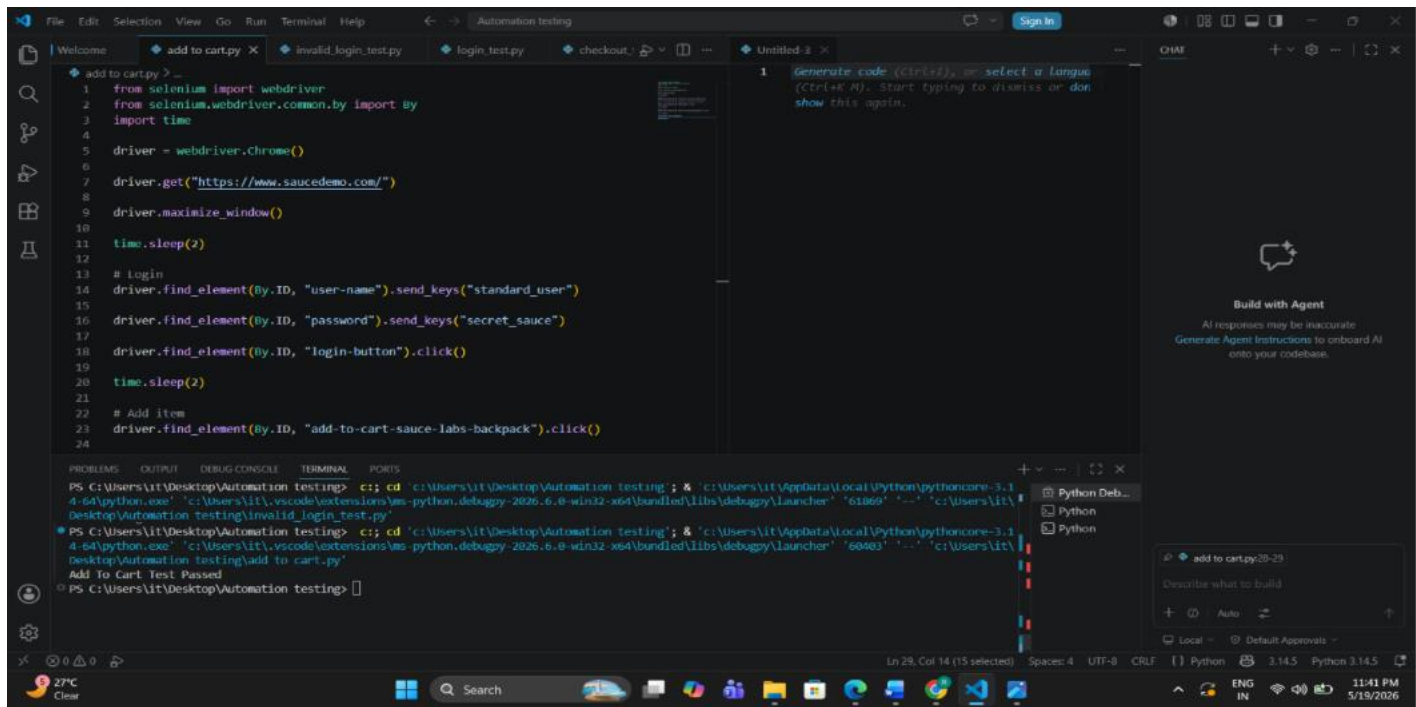
(Product inventory page)



Insert Screenshot 3 Here: (Cart icon showing added item)



(Terminal showing Add To Cart Test Passed)



4. Checkout Flow Automation Script

Purpose

To verify complete checkout functionality.

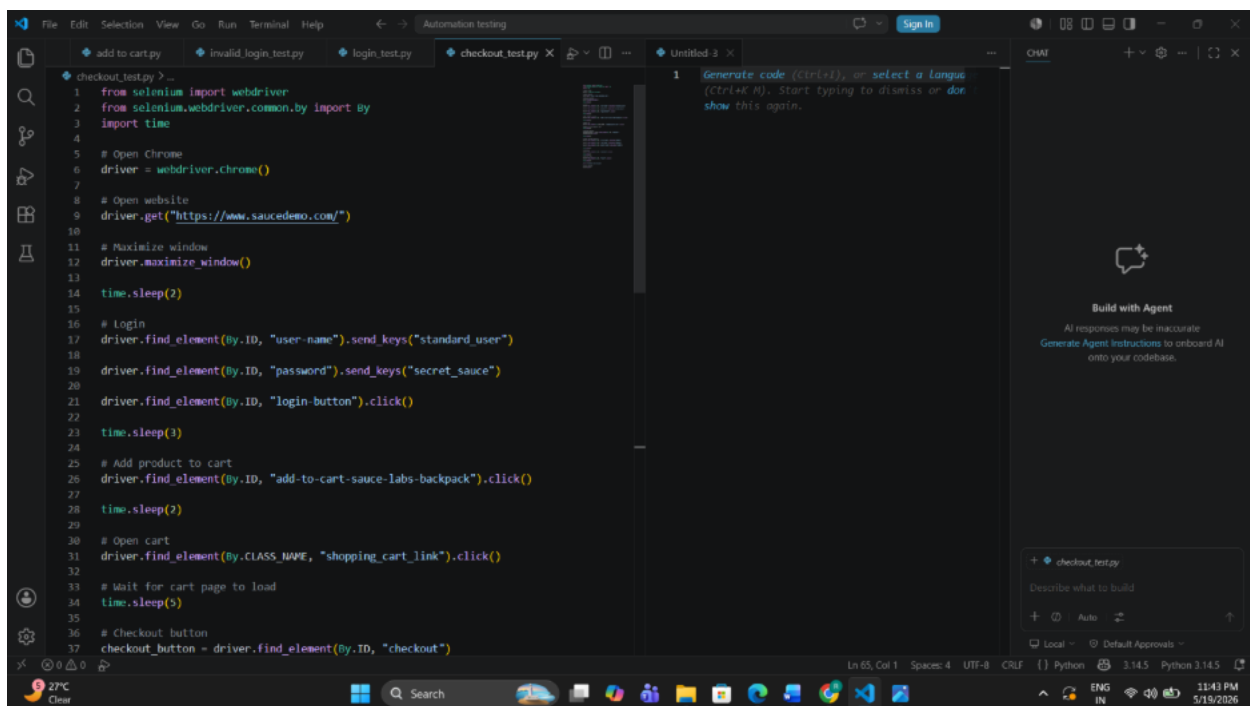
Script Validation

- Cart page loads correctly
- Checkout button functions properly
- Customer details save correctly
- Order process completes successfully
- Final confirmation page displays properly

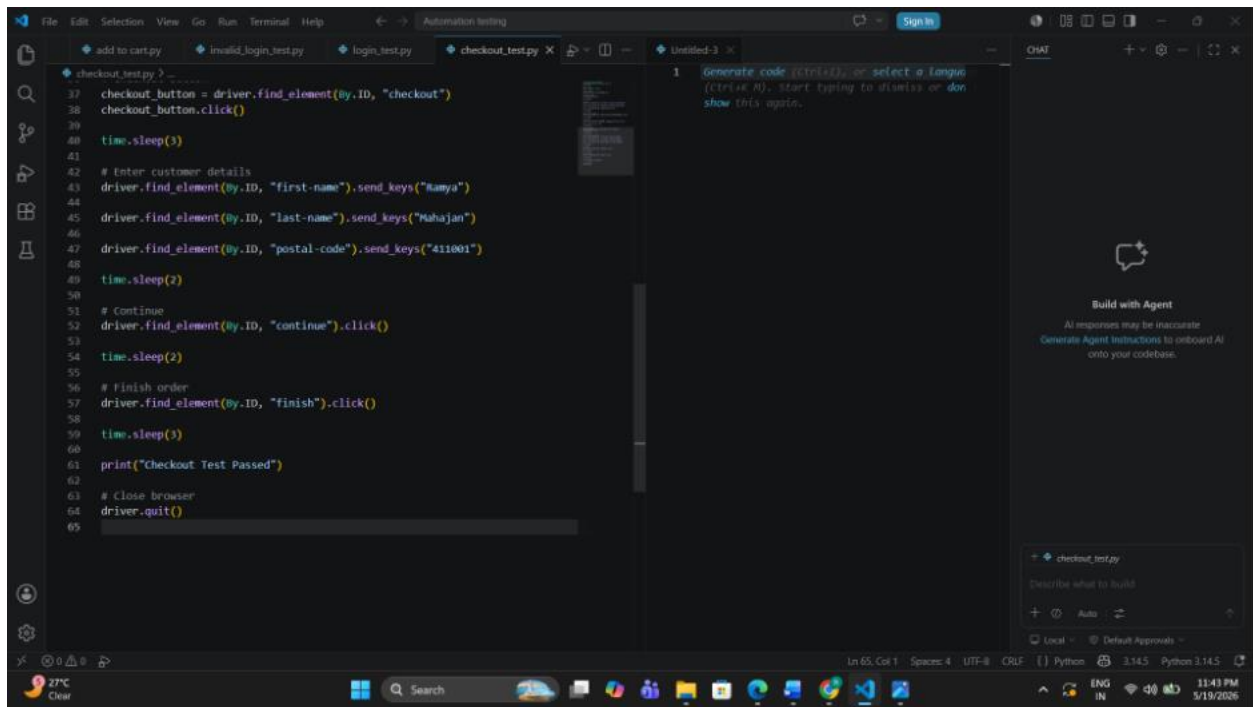
Result

Checkout process completed successfully.

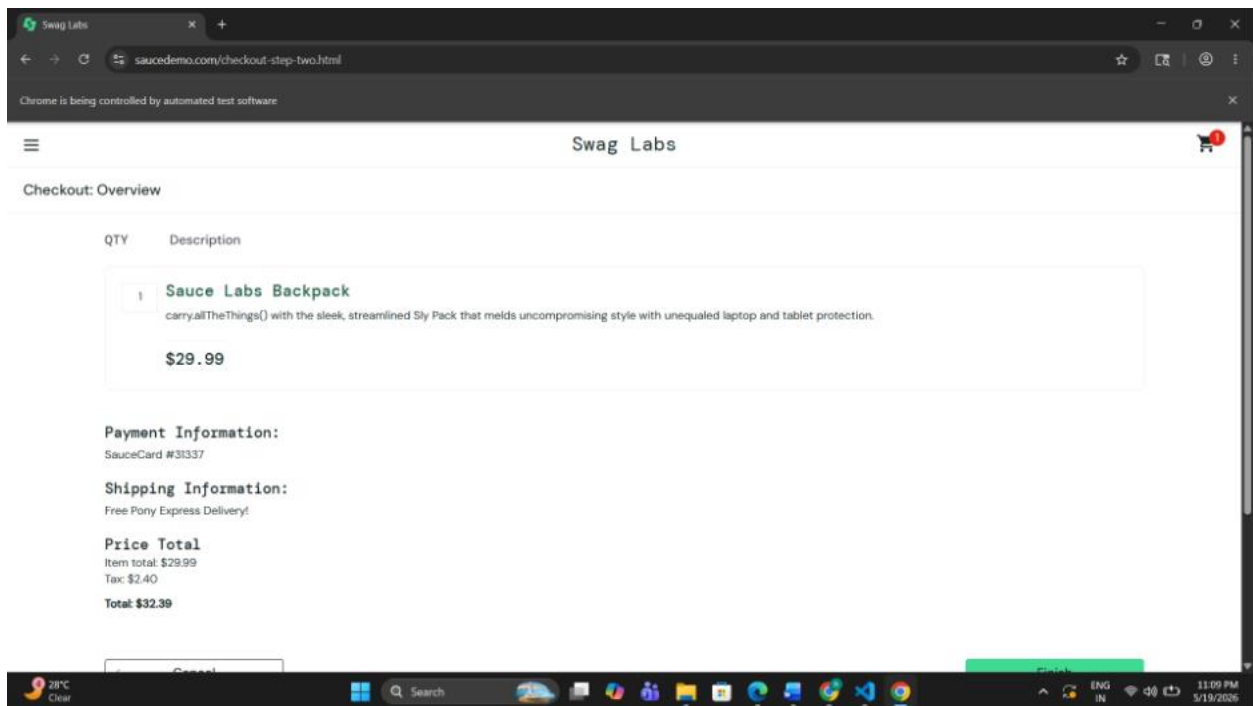
(VS Code showing checkout_test.py code)



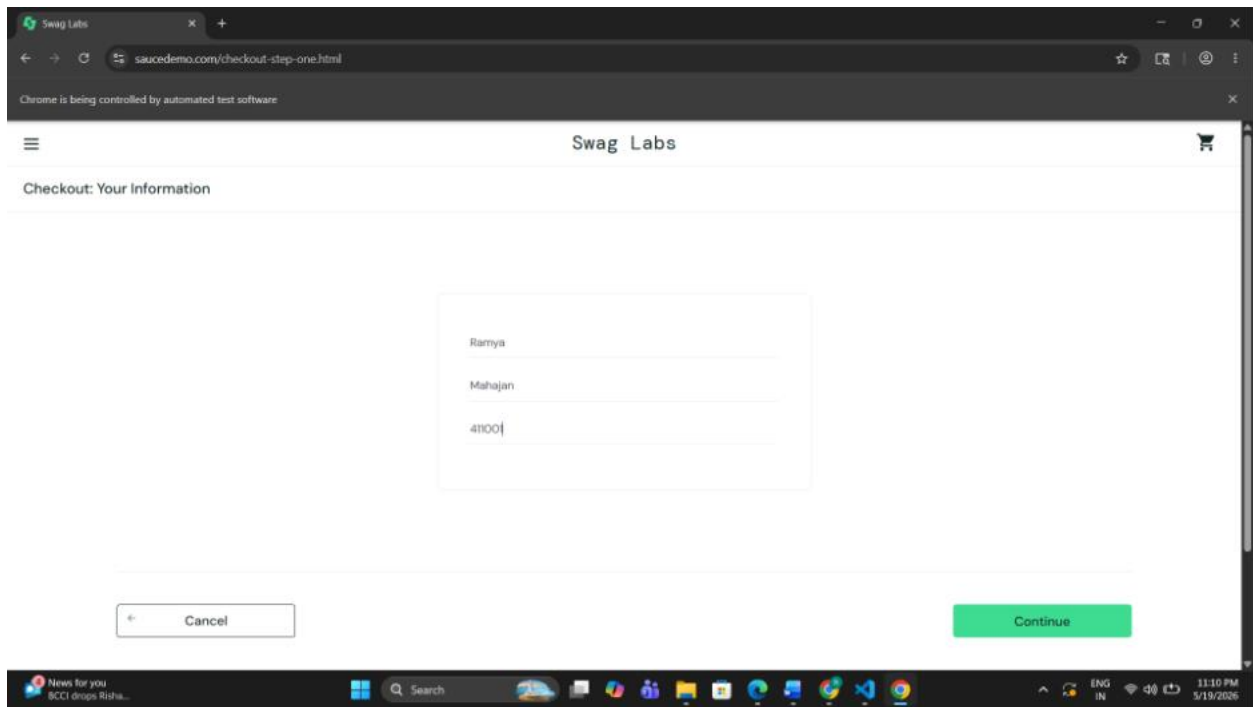
```
1 from selenium import webdriver
2 from selenium.webdriver.common.by import By
3 import time
4
5 # Open Chrome
6 driver = webdriver.Chrome()
7
8 # Open website
9 driver.get("https://www.saucedemo.com/")
10
11 # Maximize window
12 driver.maximize_window()
13
14 time.sleep(2)
15
16 # Login
17 driver.find_element(By.ID, "user-name").send_keys("standard_user")
18 driver.find_element(By.ID, "password").send_keys("secret_sauce")
19 driver.find_element(By.ID, "login-button").click()
20
21 time.sleep(3)
22
23 # Add product to cart
24 driver.find_element(By.ID, "add-to-cart-sauce-labs-backpack").click()
25
26 time.sleep(2)
27
28 # Open cart
29 driver.find_element(By.CLASS_NAME, "shopping_cart_link").click()
30
31 # Wait for cart page to load
32 time.sleep(5)
33
34 # Checkout button
35 checkout_button = driver.find_element(By.ID, "checkout")
```

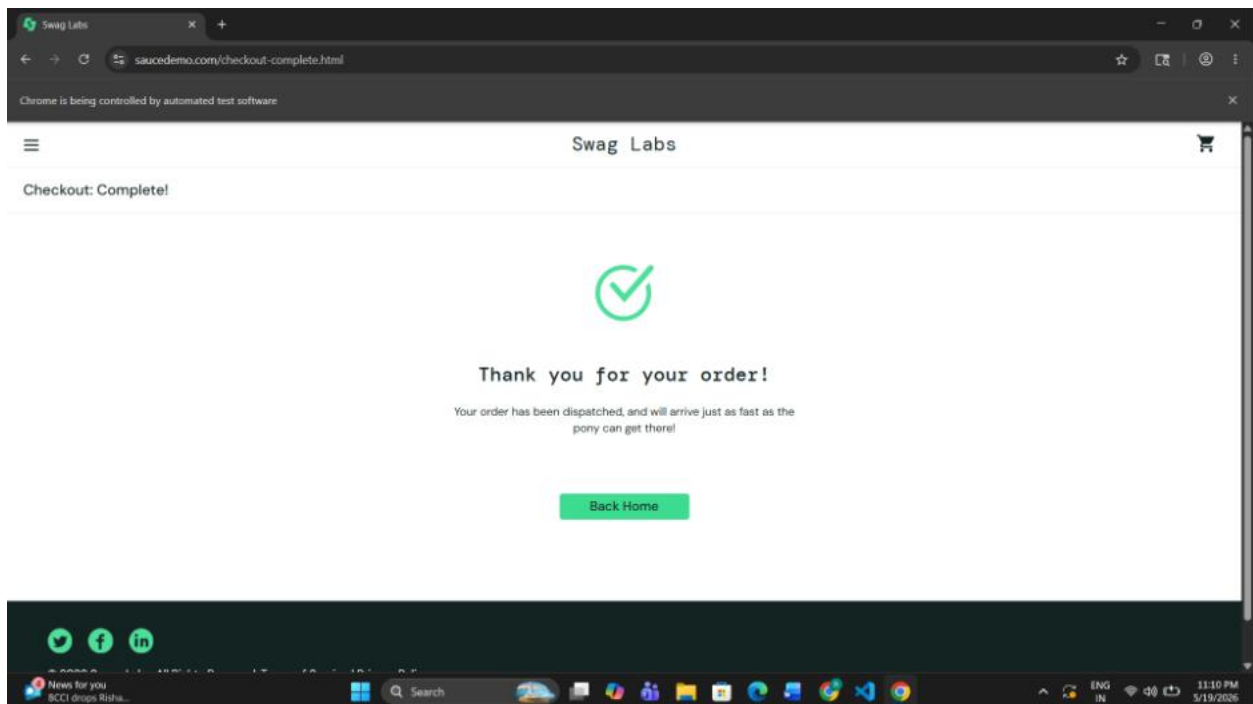
(Cart page before checkout)



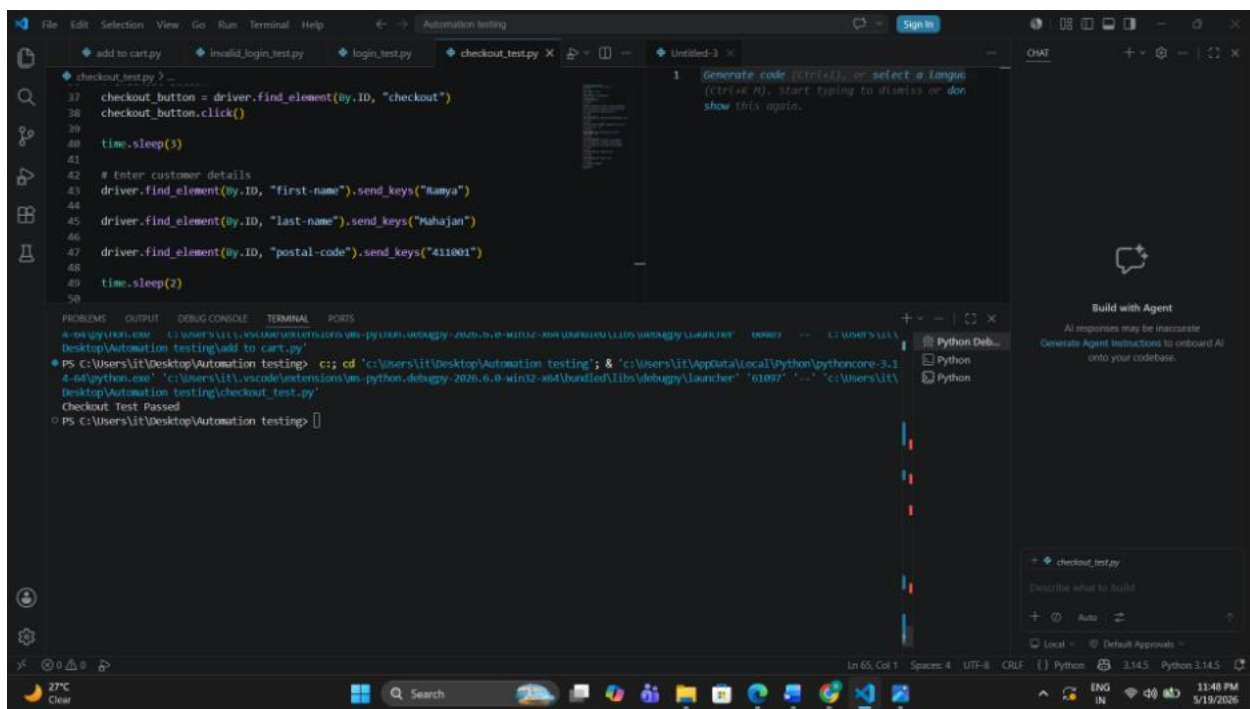
(Customer details form)



(Order confirmation / checkout completed page)



(Terminal showing Checkout Test Passed)



The screenshot shows a Visual Studio Code editor window with a file named 'checkout_test.py'. The script uses Selenium to interact with a web application. It finds a 'checkout' button, clicks it, waits for 3 seconds, and then enters customer details (name, last name, and postal code) into respective input fields, waiting 2 seconds after each entry. The terminal output shows the command to run the script and the successful execution of the test.

```
File Edit Selection View Go Run Terminal Help Automation testing
add to cart.py invalid_login_test.py login_test.py checkout_test.py X Untitled-3
1 Generate code (Ctrl+I), or select a language (Click ID), start typing to dismiss or don't show this option.

37 checkout_button = driver.find_element(By.ID, "checkout")
38 checkout_button.click()
39
40 time.sleep(3)
41
42 # Enter customer details
43 driver.find_element(By.ID, "first-name").send_keys("Ramy")
44 driver.find_element(By.ID, "last-name").send_keys("Mahajan")
45 driver.find_element(By.ID, "postal-code").send_keys("411001")
46
47 time.sleep(2)
48
49
50

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
C:\Users\it\Desktop\Automation testing> cd 'C:\Users\it\Desktop\Automation testing' & 'C:\Users\it\AppData\Local\Python\python-core-3.14.5\python.exe' 'C:\Users\it\Desktop\Automation testing\checkout_test.py'
Checkout Test Passed
PS C:\Users\it\Desktop\Automation testing>
```

Synchronization Issue Observed

During checkout automation, Selenium initially failed to locate the checkout button because the cart page required additional loading time.

Issue Identified

NoSuchElementException

Resolution Applied

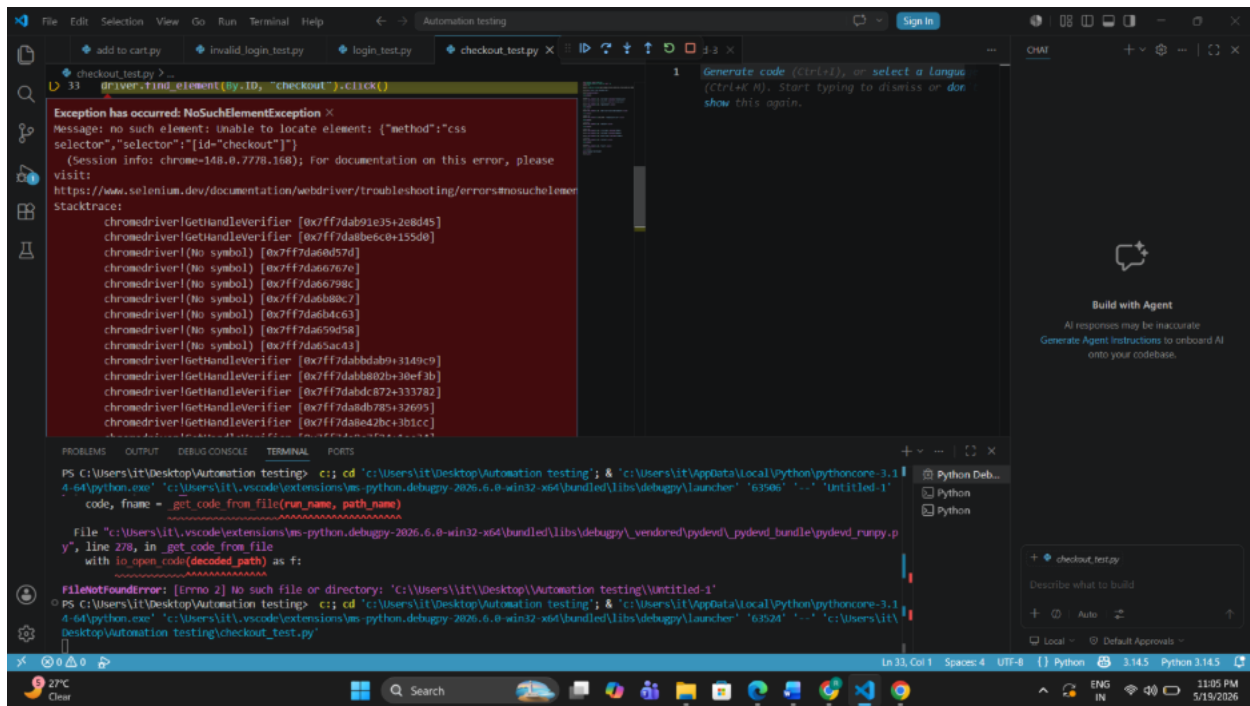
Additional wait time was added using:

```
time.sleep(5)
```

Learning Outcome

This demonstrated the importance of synchronization handling in Selenium automation testing.

(Error screen or corrected execution screenshot)



Conclusion

Automation testing improves software quality by automatically validating important application workflows after every update. Selenium automation reduces manual testing effort, increases execution speed, and helps identify issues quickly.

This practical task provided understanding of:

- Selenium WebDriver

- Python automation scripting
- Browser automation
- Functional workflow testing
- Synchronization handling
- Automated validation of application features